API 项目面试 Q&A

> 基于专家培训考试系统、电子发票网关、API网关等项目实战整理

一、高并发与限流

Q1：峰值万人同时登录，如何保证系统不崩溃？

参考回答：

我主要从三个层面做保护：

1. **接入层限流**：Redis 滑动窗口，按手机号+IP+设备维度分桶，限制单位时间内短信请求次数
2. **熔断降级**：对第三方短信服务做熔断，当失败率超过阈值时自动降级，减少对核心登录流程的影响
3. **水平扩展**：多节点部署，通过 Redis Pub/Sub 做 WebSocket 消息路由

效果：峰值期登录稳定率保持在 99% 以上（内部监控数据）

Q2：滑动窗口限流为什么比固定窗口更精准？

参考回答：

固定窗口在时间边界容易出现"突刺"——比如 59 秒和 60 秒各放行一批请求，实际上 1 秒内可能有 2 倍的流量。

滑动窗口通过 Redis ZSET 记录每个请求的时间戳，只保留窗口内的请求记录，控制更平滑。

```
// 滑动窗口实现
var now = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
var window = 60_000; // 60秒窗口
var key = $"sms:rate:{phone}";

// 移除窗口外的记录
await db.SortedSetRemoveRangeByScoreAsync(key, 0, now - window);
// 添加新请求
await db.SortedSetAddAsync(key, now, now);
await db.KeyExpireAsync(key, TimeSpan.FromMinutes(2));

var count = await db.SortedSetLengthAsync(key);
if (count > 5) return false; // 超过限制
```

Q3：短信验证码被刷，如何防控？

参考回答：

1. 频率限制：同一手机号每分钟最多 1 条、每小时最多 5 条
2. IP 限制：同一 IP 每分钟请求上限
3. 图形验证码：连续失败 3 次后触发图形验证码
4. 限流结果可配置，支持动态调整阈值

二、文件上传与转码

Q4：大文件上传如何保证弱网环境下成功率？

参考回答：

七牛云分片直传 + 断点续传：

1. **分片上传**：将大文件切分为多个小块并行上传，单个分片失败只需重传该分片
2. **断点续传**：服务端记录已上传分片列表，客户端根据 hash 校验后跳过已上传部分
3. **前端优化**：切片并行上传、失败重试、进度记忆

效果：弱网环境上传成功率提升约 30%

Q5：视频转码怎么做异步处理？

参考回答：

采用 Redis 队列 + Webhook 回调：

1. 用户上传视频后，写入转码任务到 Redis 队列，立即返回
2. 后台消费线程从队列取出任务，调用 FFmpeg 转码
3. 转码完成后，通过 Webhook 回调更新业务状态

```
        // 转码任务入队
        await redis.ListRightPushAsync("transcode:queue", JsonSerializer.Serialize(new
TranscodeJob
        {
            FileId = fileId,
            SourcePath = sourcePath,
            TargetFormat = "mp4"
        }));
```

效果：转码任务平均等待时长下降约 40%

三、WebSocket 与实时通信

Q6：多节点部署下，WebSocket 如何保证消息不丢失？

单机问题：用户 A 连接到节点 1，消息从节点 2 发起，节点 1 收不到

解决方案：Redis Pub/Sub 跨节点广播

```
        // 节点1：发布消息
        await redis.PublishAsync("ws:broadcast", JsonSerializer.Serialize(new WsMessage
        {
            UserId = userId,
            Content = content
        }));

        // 节点2：订阅并转发给本地连接
        await redis.SubscribeAsync("ws:broadcast", (channel, msg) =>
        {
            var message = JsonSerializer.Deserialize<WsMessage>(msg);
```

```
        // 查找本节点对应的连接并推送
        SendToLocalClient(message.UserId, message.Content);
    });
```

Q7：如何处理 WebSocket 连接断开与重连？

参考回答：

1. 心跳检测：客户端定时 ping，服务器超时未响应则主动关闭
2. 自动重连：前端使用指数退避策略重连
3. 消息确认：重要消息需要客户端 ACK，未确认则重发
4. 会话保持：重连后同步最新状态

四、多租户与网关

Q8：多租户数据隔离怎么做？

参考回答：

在 Admin.NET 框架中，通过租户上下文 + 数据过滤器实现：

1. 所有核心表添加 `TenantId` 字段
2. 框架层统一注入租户过滤条件，业务代码无感知
3. 敏感操作做租户校验，防止跨租户访问

```
// 租户过滤器
public class TenantFilter : ISaveFilter, IQueryFilter
{
    public void ApplyFilter(TEntity entity)
    {
        if (entity is ITenantEntity tenantEntity)
            tenantEntity.TenantId = CurrentTenant.Id;
    }
}
```

Q9：API 网关如何统一鉴权？

参考回答：

1. 统一入口：所有请求经过网关
2. 令牌校验：验证 JWT Token 有效性
3. 权限透传：将用户信息、角色、租户等上下文放入请求头转发给后端
4. 限流熔断：对异常请求做限流和熔断

```
// Refit 接口代理
[Headers("Authorization: Bearer")]
public interface IBackendApi
{
    [Get("/api/data")]
    Task<DataResponse> GetData([Header("X-Tenant-Id")] string tenantId);
}
```

五、第三方集成

Q10：多个开票渠道接口不同，如何统一管理？

参考回答：

工厂模式 + 接口抽象：

1. 定义统一接口 `IInvoiceClient`
2. 每个厂商实现独立类
3. 工厂根据配置创建对应实例

```
public interface IInvoiceClient
{
    Task<InvoiceResult> IssueAsync(InvoiceRequest request);
    Task<InvoiceResult> QueryAsync(string invoiceNo);
}

public class VendorAClient : IInvoiceClient { /* ... */ }
public class VendorBClient : IInvoiceClient { /* ... */ }

public class InvoiceClientFactory
{
    public IInvoiceClient Create(string vendor) => vendor switch
    {
        "A" => new VendorAClient(),
        "B" => new VendorBClient(),
        _ => throw new NotSupportedException()
    };
}
```

效果：新接入开票渠道只需新增实现类，核心代码零改动

Q11：第三方接口调用频繁，如何优化？

参考回答：

MongoDB TTL 缓存：

1. 缓存第三方返回的结构化数据
2. 设置 TTL 索引自动过期（如 5 分钟）
3. 相同请求优先查缓存

```
// MongoDB TTL 索引
db.thirdPartyCache.createIndex(
    { "expireAt": 1 },
    { expireAfterSeconds: 0 }
)
```

效果：第三方 API 调用次数下降约 50%

六、设计模式应用

Q12：工厂模式和策略模式有什么区别？

模式	目的	场景
工厂模式	封装对象创建	创建逻辑复杂、需根据配置创建不同实现
策略模式	封装算法/行为	运行时切换不同算法、业务规则分支多

项目中应用：

- 工厂模式：电子发票网关创建不同厂商的客户端
- 策略模式：金融保函根据资方不同选择不同风控策略

七、支付/金融安全

Q13：金融保函系统如何保证报文安全？

参考回答：

1. **SM4 对称加密**：报文内容加密传输
2. **摘要校验**：MD5/SM3 摘要防篡改
3. **签名验证**：请求方签名，接收方校验
4. **审计日志**：记录所有报文流水，便于追溯

```
// 报文签名
var signContent = $"body={json}×tamp={timestamp}&key={secret}";
var sign = SM3(signContent);
```

八、面试反问准备

Q14：你有什么问题想问面试官的？

建议问题：

1. 贵司后端技术栈主要是什么？是否有 API 网关、统一认证中心？
2. 这个岗位主要负责哪个业务模块？需要对接哪些第三方服务？
3. 团队目前的并发规模大概是多少？有没有遇到什么技术挑战？
4. 有没有代码规范、Code Review、自动化测试等工程实践？

建议：结合简历中的项目，选择 2-3 个最深入的场景重点准备，确保能回答出"怎么做的"和"为什么这样做"